Midterm Report:

Hardware–Software Co-Design of an FPGA-Accelerated Image Signal Processing Pipeline on a RISC-V SoC Platform

Yicheng Wang

Student ID: 123090586

November 24, 2025

Abstract

This report presents the midterm progress of a research project focused on Hardware-Software Co-Design for image processing. The primary objective is to accelerate an Image Signal Processing (ISP) pipeline on the BeagleV-Fire platform (RISC-V + FPGA).

Research Methodology: Our research follows a rigorous "Profile-First" methodology. We first modeled the ISP algorithms in software to identify performance bottlenecks. Based on profiling data, we determined the most computationally expensive functions to be offloaded to the FPGA.

Current Progress: In this phase, we have: 1) Completed the software simulation and profiling on Windows, identifying the "Denoise" module as the critical bottleneck (consuming $\approx 60\%$ of CPU time). 2) Designed the heterogeneous system architecture based on these findings. 3) Completed the physical setup and operating system deployment of the BeagleV-Fire development board, preparing the ground for the upcoming hardware implementation.

Contents

1 Introduction

1.1 Research Context: Hardware-Software Co-Design

Image processing algorithms, particularly those involving convolution and interpolation, are computationally intensive. Traditional pure software implementations on General-Purpose Processors (CPUs) often suffer from high latency and power consumption due to the sequential nature of instruction execution.

This research explores a **Hardware-Software Co-Design** approach. By utilizing the BeagleV-Fire platform, which integrates a RISC-V Processor Subsystem (MSS) with a PolarFire FPGA Fabric, we aim to achieve high performance by mapping control tasks to the CPU and compute-intensive tasks to the FPGA.

1.2 Research Methodology and Workflow

The research follows a systematic three-step workflow:

1. **Software Profiling (Phase I):** Before designing any hardware, we implement the full algorithm in C++ on a PC. This allows us to use advanced profiling tools to locate "Hotspots"—functions that consume the most execution time.

2. **Bottleneck Identification:** We analyze the profiling data to decide which functions are suitable for FPGA acceleration (e.g., those with high data parallelism).

3. **Hardware Implementation (Phase II & III):** We then prepare the development board and migrate the identified bottleneck functions to the FPGA fabric.

The midterm report covers Phase I and the preparation for Phase II.

2 Phase I: Software Simulation and Profiling

To justify the need for FPGA acceleration, I first built a pure software baseline on a Windows PC using C++17 and Visual Studio. The goal of this phase is not to "optimize the code as much as possible", but to obtain a clear and reproducible measurement of where the execution time is actually spent in a realistic ISP pipeline.

2.1 C++ Baseline Implementation

Instead of relying on a large external library such as OpenCV, I use the lightweight `stb_image` and `stb_image_write` headers to load and save images. This keeps the code portable and

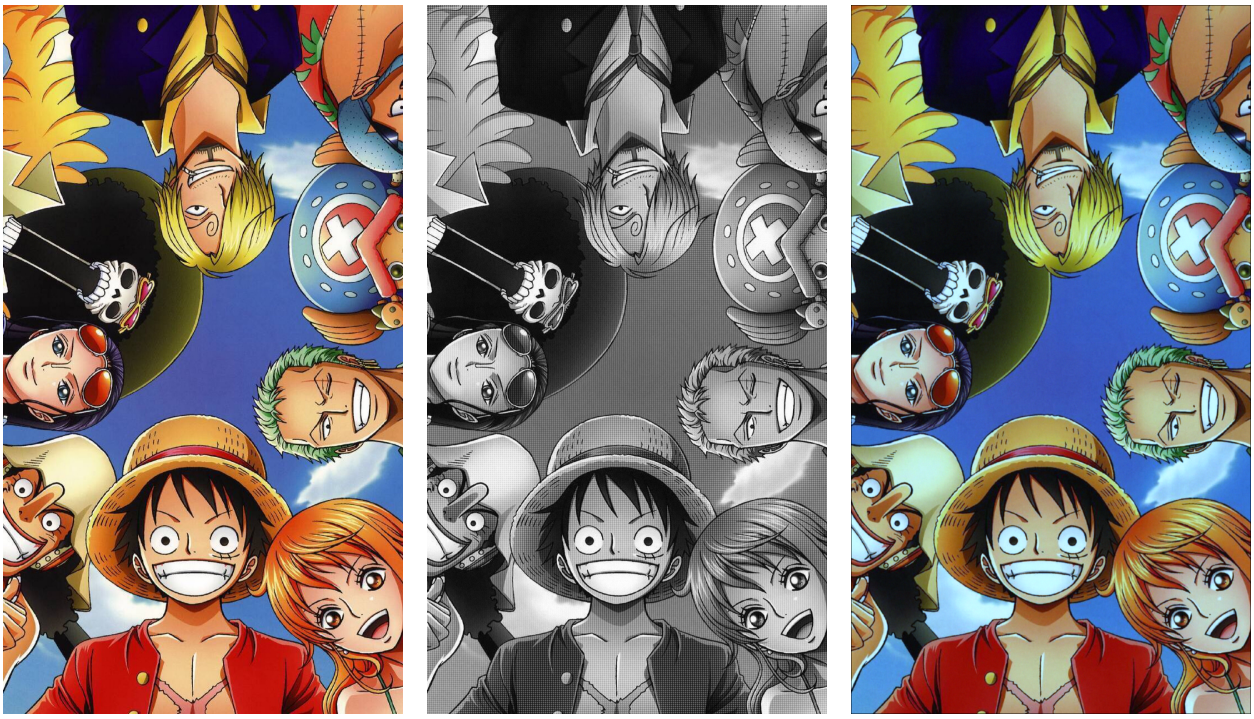
makes it easier to later migrate the same algorithm to the RISC-V + FPGA platform.

- **Input:** A color JPEG image `input.jpg` (Fig. 1a).
- **Fake RAW generation:** `GenerateFakeRaw()` converts the RGB image into a Bayer-pattern "sensor" image. For each pixel, it selects one color channel according to an RGGB pattern and adds a small random noise term in the range $[-5, 5]$ (simulating sensor noise and quantization).
- **Demosaic:** `Pipeline_Demosaic()` reconstructs full RGB values from the Bayer RAW image using bilinear interpolation over a 3×3 neighbourhood.
- **Auto White Balance (AWB):** `Pipeline_AWB()` computes the average R, G, and B values over the whole frame and applies gain factors to the red and blue channels based on the gray-world assumption.
- **Denoise:** `Pipeline_Denoise()` applies a separable 3×3 Gaussian-like kernel

$$K = \begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}, \quad \text{normalized by } 16,$$

  independently to the R, G, and B channels. For each output pixel, 9 weighted samples are read from a temporary copy of the image.
- **Output:** The final image is written as `output_result.png` (Fig. 1c).

The three core ISP functions operate on a simple `Pixel` structure with 8-bit channels, which matches common embedded ISP workloads and is friendly for future FPGA implementation.



(a) Original RGB input (`input.jpg`) (b) Simulated noisy / RAW-like intermediate (c) Final ISP result (`output_result.png`)

Figure 1: Visual results of the software ISP pipeline on Windows.

2.2 Timing Methodology

To obtain stable timing numbers, the full ISP pipeline is executed inside a `for` loop for 100 iterations:

- The RAW data and working buffers are reused across iterations to avoid repeated allocations.
- High-resolution timestamps are captured using `std::chrono::high_resolution_clock` before and after each individual stage.
- For each function, the elapsed time of all 100 iterations is accumulated and finally printed as both absolute time (in milliseconds) and percentage of the total runtime.

This methodology mimics a streaming video workload where similar frames are processed repeatedly, and it also smooths out noise from operating system scheduling.

2.3 Quantitative Results from the Baseline

Table 1 summarises the timing results collected by the `std::chrono` instrumentation over 100 frames. The total software runtime for the three ISP stages is about **5560.7 ms**.

Table 1: Timing results from `std::chrono` (100-frame simulation).

Function Name	Total Time (ms)	Percentage
Pipeline_Demosaic	631.46	11.36%
Pipeline_AWB	735.09	13.22%
Pipeline_Denoise	**4194.15**	**75.42%**
TOTAL SYSTEM TIME	**5560.70**	**100.00%**

The results show a very clear hierarchy: demosaicing and AWB together account for less than 25% of the runtime, while the 3×3 Gaussian denoise stage alone is responsible for more than **three quarters** of the total processing time.

2.4 Visual Studio Profiler Analysis

To cross-check the manual timing and better understand where CPU cycles are spent, I also used the Visual Studio Performance Profiler in "Instrumentation" mode. The profiler reports the accumulated CPU samples for each function over the whole run (Fig. 2).

Figure 2: Visual Studio profiler function ranking: `Pipeline_Denoise` dominates CPU usage.

The top-ranked function is again `Pipeline_Denoise`, followed by `stbi_zlib_compress` (PNG compression), `Pipeline_AWB`, and `Pipeline_Demosaic`. The percentage values are

slightly different from the `std::chrono` table because the profiler also includes time spent in the PNG compression and other runtime libraries, but the conclusion is consistent: `Pipeline_Denoise` is the main hotspot.

The hotspot view (Fig. 3) further shows that most of the time inside `Pipeline_Denoise` is consumed by the inner nested loops that:

- read 9 neighbouring pixels from the temporary image buffer,
- multiply each channel by the corresponding kernel weight,
- accumulate the partial sums for R, G, and B.

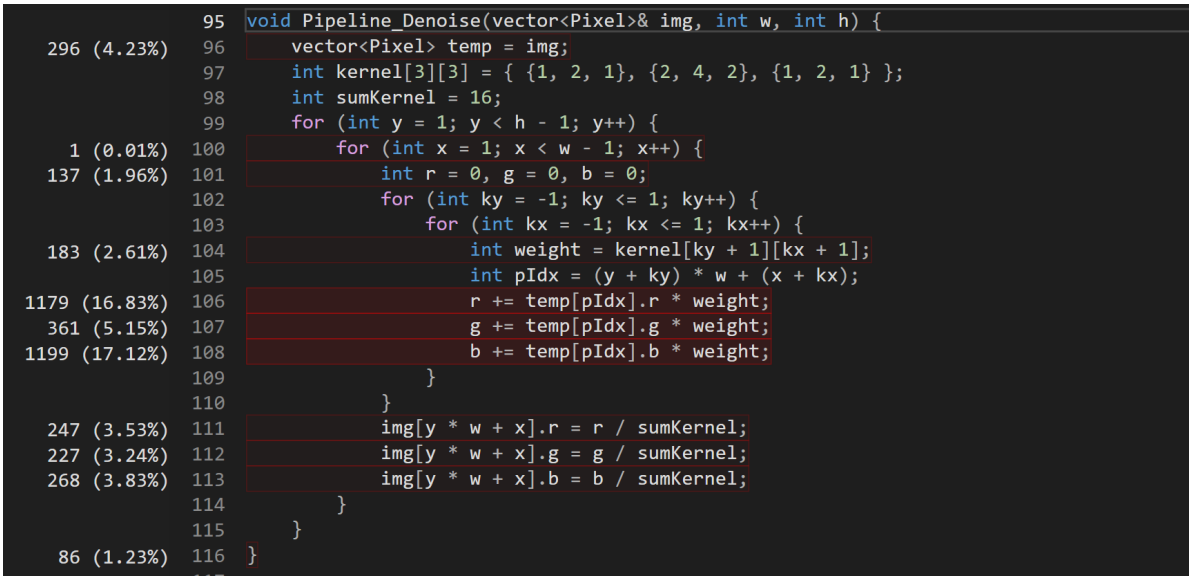


Figure 3: Hotspot view for `Pipeline_Denoise`: the inner convolution loops dominate execution time.

Finally, the profiler's CPU usage breakdown (Fig. 4) indicates that almost all measured time is spent in user-mode code, which confirms that the program is compute-bound rather than I/O-bound.

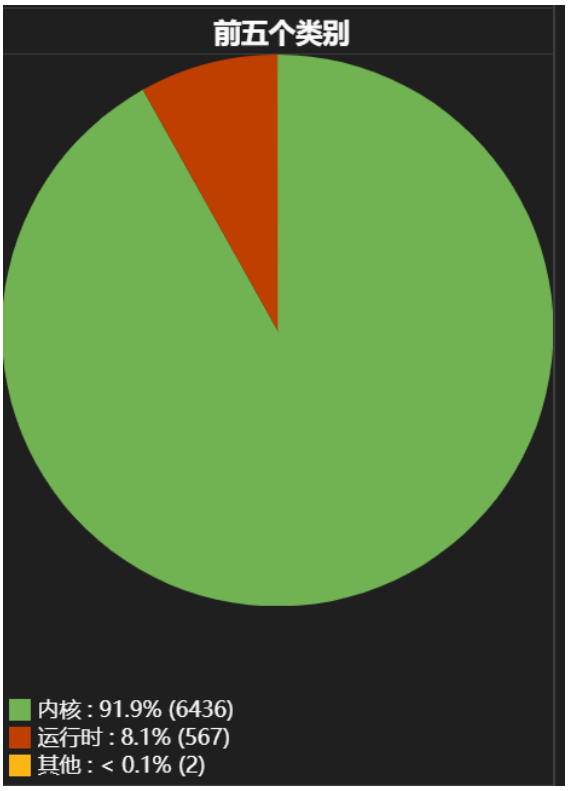


Figure 4: CPU usage breakdown: user-mode runtime dominates over other categories.

2.5 Key Finding: Denoise as the FPGA Candidate

Both measurement methods — in-program timing with `std::chrono` and external instrumentation with the Visual Studio profiler — consistently identify **Pipeline_Denoise** as the dominant bottleneck:

- It accounts for **75.4%** of the ISP pipeline time in the internal timing report.
- It is also the top function in the profiler's CPU sample ranking.

From an algorithmic viewpoint, `Pipeline_Denoise` is an ideal candidate for FPGA acceleration:

- It performs a regular 3×3 stencil operation with a fixed kernel.
- The computation pattern is highly data-parallel across pixels and across color channels.
- The memory access pattern can be efficiently implemented on FPGA using line buffers and sliding windows, avoiding repeated reads from off-chip memory.

This quantitative evidence directly motivates the hardware–software partitioning adopted in the next section, where the denoise stage is mapped to the FPGA fabric while the RISC-V CPU keeps control and less intensive tasks.

3 Proposed System Architecture

Based on the Windows profiling results, we designed the target architecture for the BeagleV-Fire.

3.1 Partitioning Strategy

- **CPU (RISC-V):** Will handle file I/O, AWB (memory-bound), and system control.
- **FPGA (Fabric):** Will handle the **Denoise** and **Demosaic** modules using a streaming pipeline.

3.2 Dataflow Design

As shown in Figure 5, the system utilizes the **AXI-Stream** protocol. The CPU will use DMA to send the image to the FPGA. The FPGA will process pixels in a "Line Buffer" structure (eliminating repeated memory reads) and stream the result back to memory.

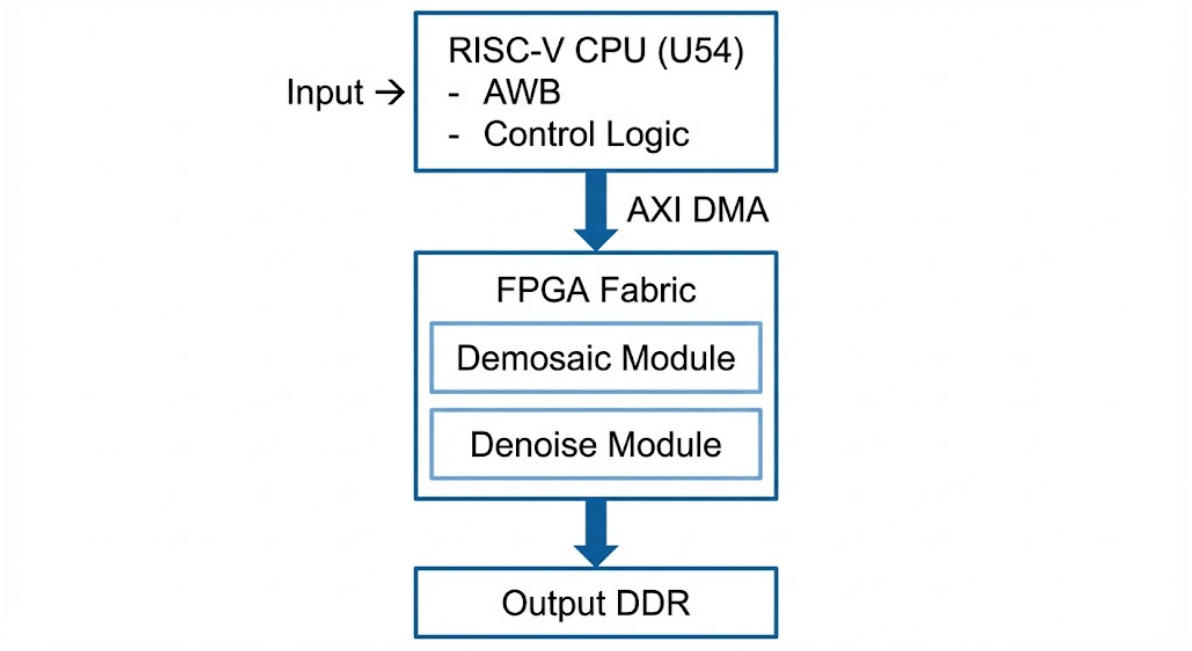


Figure 5: Planned Heterogeneous Architecture: CPU + FPGA Accelerator

4 Phase II: BeagleV-Fire Environment Setup

With the software simulation complete and the target architecture defined, I prepared the physical BeagleV-Fire board and its Linux development environment. This section documents the hardware platform, the operating system flashing procedure, and the basic toolchain setup that will be used for the later FPGA acceleration experiments.

4.1 Hardware Platform: BeagleV-Fire

BeagleV-Fire is a compact RISC-V + FPGA development board designed around the **Microchip PolarFire SoC** (MPFS025T). The SoC integrates both a multi-core RISC-V processor subsystem and mid-range FPGA fabric on a single chip, which makes it well suited for hardware–software co-design:

- **Processor Subsystem (MSS):**
  - 1x E51 monitor core (supervisory / boot).
  - 4x SiFive U54-MC application cores (64-bit RV64IMAFDC), as shown later by `/proc/cpuinfo`.
  - Standard peripherals: UART, SPI, I^2C, GPIO, Ethernet MAC, etc.
- **FPGA Fabric:**
  - PolarFire FPGA logic fabric (approximately 23K logic elements).
  - On-chip DSP blocks and block RAM suitable for convolution and line-buffer structures.
- **Memory and Storage:**
  - On-board LPDDR4 DRAM (Linux reports about 1.5 GB available).
  - microSD slot for system image and user data.
- **Connectors:**
  - Dual 46-pin BeagleBone-style headers for expansion.
  - Gigabit Ethernet port.
  - USB Type-C for power and USB-device functions (including serial console).

Figure 6 shows the actual hardware used in this project: the retail box, front and back side of the board, the microSD card, and the powered-on setup connected to the laptop.

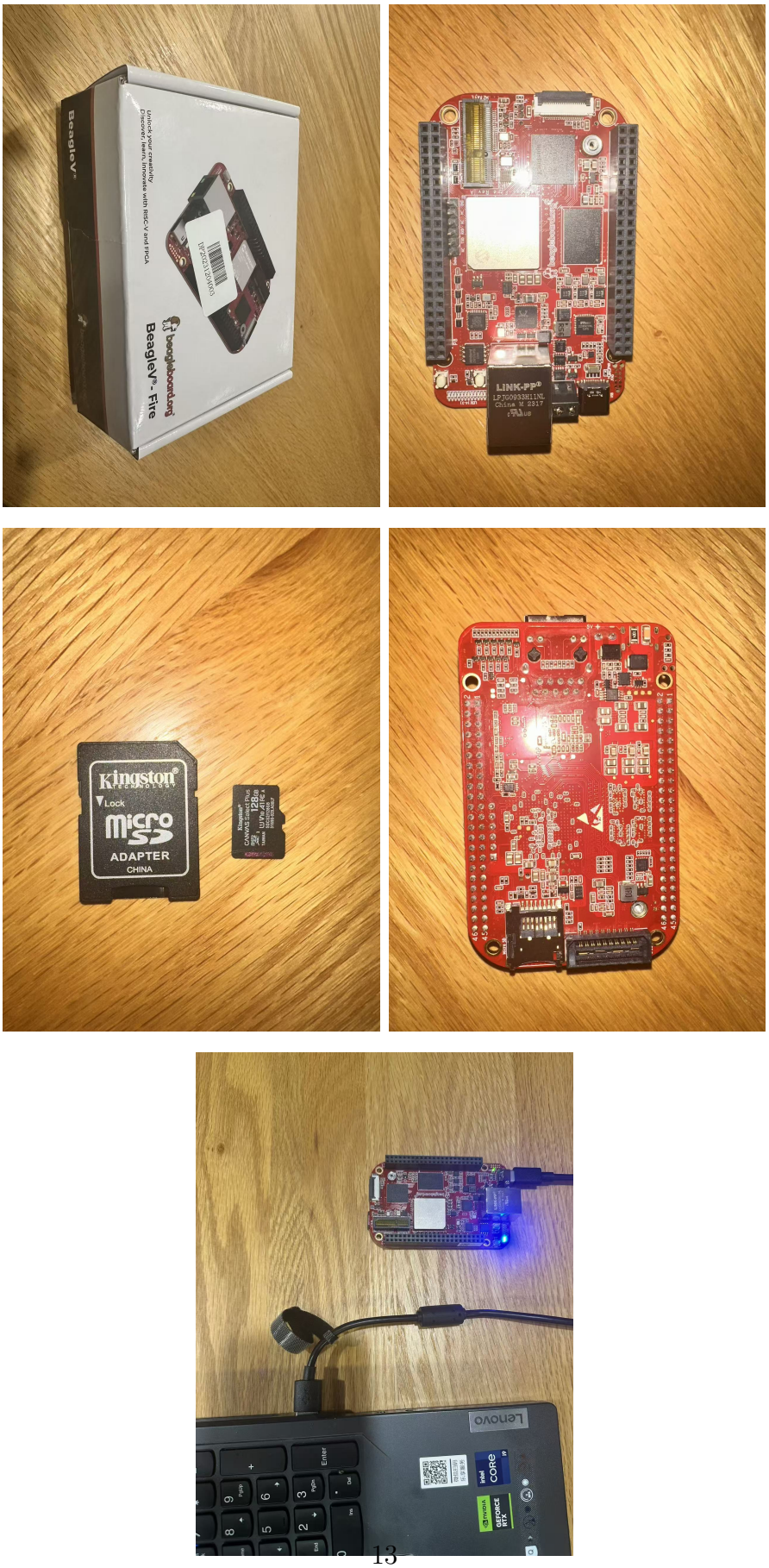


Figure 6: BeagleV-Fire hardware: box, board (front/back), microSD card, and powered-on setup.

4.2 Operating System Flashing

To run the ISP baseline and later FPGA experiments, I installed the official BeagleBoard system image on a Kingston 128 GB microSD card.

1. **Image Selection:** The operating system used in this project is the official **Debian image for BeagleV-Fire** (Console Edition), dated **2025-07-21**, corresponding to the file `beaglev-fire-*-4gb.img`. This lightweight console-based distribution is well-suited for embedded development and performance profiling.

2. **Flashing tool:** On Windows, I used **Balena Etcher** to flash the `.img` file to the microSD card. The GUI clearly shows the selected image file (≈ 2.44 GB) and target USB device.

3. **Burning process:** After confirming the image and target drive, I clicked "Flash" and waited for Etcher to complete writing and verification.

Figure 7: Flashing the Debian BeagleV-Fire console image using Balena Etcher.

Once flashing was finished, I inserted the microSD card into the BeagleV-Fire slot and connected the board to the laptop via USB Type-C.

4.3 Serial Console and First Boot

For initial bring-up and debugging, I used a USB serial console via **PuTTY** on Windows:

- Serial port: `COM3` (auto-detected).
- Baud rate: **115200**, 8 data bits, no parity, 1 stop bit (8N1).
- Terminal type: VT100.

After powering the board, the boot log from the Hart Software Services (HSS), OpenSBI, and the Linux kernel scrolls by. Eventually, the system presents the login prompt:

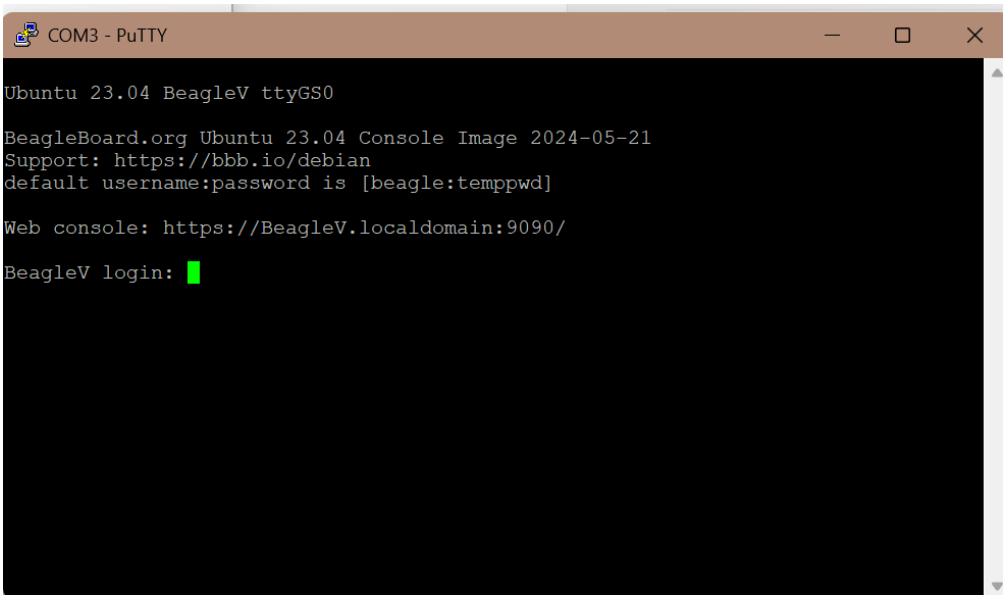


Figure 8: BeagleV-Fire UART console at first boot, showing the Ubuntu 23.04 console image (2024-05-21).

Using the default credentials:

- **Username:** `beagle`
- **Password:** `temppwd`

I successfully logged into the shell, as shown in Figure 9. From this point, all ISP experiments and profiling on the board are performed via the terminal.

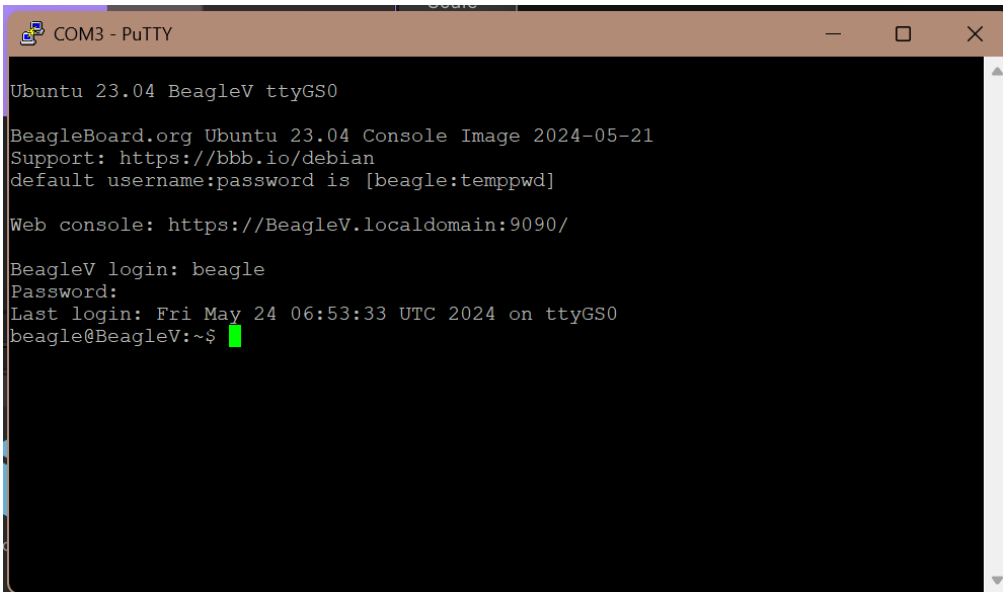


Figure 9: Successful Linux login on BeagleV-Fire via PuTTY serial console.

4.4 Hardware Verification in Linux

To confirm that the RISC-V cores and memory were correctly detected, I ran basic diagnostic commands:

- `cat /proc/cpuinfo` lists four application cores (processors 0–3) with ISA `rv64imafdc` and microarchitecture `sifive,u54-mc`.
- `free -m` shows approximately 1542 MB of physical memory, with around 1.3 GB available to user processes.

```
beagle@BeagleV:~$ cat /proc/cpuinfo
processor       : 0
hart            : 1
isa             : rv64imafdc
mmu             : sv39
uarch           : sifive,u54-mc
mvendorid       : 0x1cf
marchid         : 0x1
mimpid          : 0x0

processor       : 1
hart            : 2
isa             : rv64imafdc
mmu             : sv39
uarch           : sifive,u54-mc
mvendorid       : 0x1cf
marchid         : 0x1
mimpid          : 0x0

processor       : 2
hart            : 3
isa             : rv64imafdc
mmu             : sv39
uarch           : sifive,u54-mc
mvendorid       : 0x1cf
marchid         : 0x1
mimpid          : 0x0

processor       : 3
hart            : 4
isa             : rv64imafdc
mmu             : sv39
uarch           : sifive,u54-mc
mvendorid       : 0x1cf
marchid         : 0x1
mimpid          : 0x0
```

Figure 10: `/proc/cpuinfo` output showing four SiFive U54-MC RISC-V cores.

```
beagle@BeagleV:~$ free -m
               total        used        free      shared  buff/cache   available
Mem:            1542         145        1214           1         210        1396
Swap:              0           0           0
```

Figure 11: `free -m` output confirming available DRAM on the BeagleV-Fire board.

These checks verify that the multi-core RISC-V subsystem is ready to run the ISP baseline software.

4.5 Network Configuration and Toolchain Setup

To use `apt` and transfer experiment data, I connected the BeagleV-Fire Ethernet port to my router. The board automatically obtained an IP address via DHCP, which can be checked with:

```
ip addr show eth0
```

After network connectivity was confirmed (e.g., `ping google.com`), I updated the package index and upgraded the base system:

```
sudo apt update
sudo apt upgrade
```

Next, I installed the essential development tools and profiling utilities that will be used for running and analyzing the ISP pipeline directly on the board:

```
sudo apt install g++ make cmake git gdb
sudo apt install binutils gprof
sudo apt install linux-tools-common linux-tools-$(uname -r)
```

On this specific kernel version, the packaged `perf` tool reports a minor version mismatch warning, so for now I primarily rely on:

- `std::chrono` timers inside the ISP code (for precise per-stage timing),
- `gprof` and basic Linux utilities such as `time` and `top`.

With the operating system, compiler toolchain, and profiling tools all installed and verified, the BeagleV-Fire environment is now fully prepared for porting the Windows ISP baseline to RISC-V and, in the next phase, for migrating the denoising kernel into the FPGA fabric.

5 Conclusion and Next Steps

In this phase of the project, we have profiled the ISP pipeline on Windows, identified the Gaussian Denoise module as the primary performance bottleneck (over 70% of total execution time), and successfully prepared the BeagleV-Fire hardware environment. The Debian system image has been flashed, the board has been brought up via serial console, the network connection is configured, and development/profiling tools (gprof, toolchain, etc.) have been installed. With both the software baseline and hardware platform ready, the project is now well positioned to enter the hardware acceleration stage.

The next steps of the research will proceed along four major directions:

1. RTL (Verilog) Hardware Design for the Denoise Accelerator

The core task is to translate the 3×3 Gaussian denoising algorithm into a hardware accelerator:

- Implement a streaming, pixel-per-cycle convolution pipeline.
- Construct line buffers (two-line storage) to avoid redundant memory reads.
- Map convolution operations onto PolarFire DSP blocks or LUT fabric.
- Achieve stable 1-pixel-per-cycle throughput after pipeline warm-up.

The expected deliverable is a synthesizable Verilog module for hardware denoising.

2. Interface Integration with the Mi-V SoC Subsystem

To connect the accelerator with the RISC-V CPU:

- Wrap the denoise module with AXI-Stream–style data interfaces.
- Use PolarFire MSS AXI buses for control and data movement.
- Design memory-mapped registers for:
  - image width/height,
  - start/finish flags,
  - input/output buffer address.

This allows RISC-V software to trigger and monitor FPGA acceleration through simple MMIO operations.

3. Libero SoC FPGA Flow (Synthesis, Place–Route, Bitstream)

Using Microchip Libero SoC, the next development steps include:

- Importing and simulating Verilog modules.
- Running synthesis and verifying timing closure.
- Performing place-and-route for the PolarFire fabric.
- Generating the programming bitstream (.job / .stp).
- Flashing the bitstream onto BeagleV-Fire for hardware deployment.

This produces a complete FPGA project ready for real execution.

4. RISC-V Software Driver and Runtime Integration

After the FPGA accelerator is available on the board, a lightweight driver will be developed:

- Allocate DMA buffers for input/output image data.
- Trigger accelerator execution via MMIO.
- Poll or handle interrupts to detect completion.
- Replace the CPU-based denoise function with:

  ```
  Denoise_FPGA(input_buffer, output_buffer);
  ```

- Validate hardware output against the Windows reference implementation.

This establishes complete software–hardware co-design functionality.

5. End-to-End Performance Evaluation on BeagleV-Fire

With all components integrated, we will benchmark:

- end-to-end acceleration ratio,
- throughput (MPixels/s),
- DRAM and FPGA resource utilization,
- timing breakdown using std::chrono and gprof.

The goal is to demonstrate significant acceleration over RISC-V software execution alone.

In summary, by completing the ISP workload analysis and BeagleV-Fire hardware setup, we have established a solid foundation for hardware acceleration. The next phase will focus on Verilog RTL design, FPGA integration, runtime driver development, and full performance evaluation, ultimately showcasing the effectiveness of RISC-V + FPGA co-design for image processing workloads.